



INTERNACIONAL TRAVESSIAS SALVADOR

ITS-Demandas

Guia de Implantação em Servidor / VM

Sumário

1. Introdução	4
1.1 Objetivo deste documento	4
1.2 Visão geral da arquitetura de implantação	4
1.3 Opções de implantação	4
2. Pré-requisitos	6
2.1 Hardware recomendado para a VM	6
2.2 Software necessário	6
2.3 Rede e domínio	6
2.4 Conta Supabase	7
3. Preparando o Banco de Dados	8
3.1 Criando o projeto no Supabase	8
3.2 Rodando o schema	8
3.3 Coletando as chaves de API	8
3.4 Criando o usuário administrador	9
4. Provisionando o Servidor	10
4.1 Escolhendo a VM	10
4.2 Acesso inicial e hardening básico	10
5. Instalando Dependências no Servidor	11
5.1 Node.js	11
5.2 Git	11
5.3 PM2	11
5.4 Nginx	11
6. Implantando a Aplicação	12
6.1 Transferindo o código	12

6.2 Variáveis de ambiente	12
6.3 Instalação e build	13
6.4 Rodando com PM2	13
6.5 Alternativa: serviço systemd	13
7. Proxy Reverso e HTTPS	15
7.1 Configurando o Nginx	15
7.2 Certificado SSL com Let's Encrypt	15
8. Segurança	17
8.1 Firewall	17
8.2 Segredos e variáveis sensíveis	17
8.3 Atualizações do sistema operacional	17
8.4 Backups	17
9. Alternativa: Implantação com Docker	18
10. Processo de Atualização	20
11. Monitoramento e Manutenção	21
12. Checklist Pós-Implantação	22
13. Solução de Problemas	23
14. Anexo: Referência de Variáveis de Ambiente	24

1. Introdução

1.1 Objetivo deste documento

Este guia descreve, passo a passo, como colocar o ITS-Demandas para rodar em um servidor ou máquina virtual (VM) próprios da empresa — cenário típico de um ambiente corporativo que prefere manter a aplicação dentro da própria infraestrutura, em vez de depender de uma plataforma de hospedagem externa. Ele assume que quem for executar os passos tem familiaridade básica com linha de comando em Linux.

1.2 Visão geral da arquitetura de implantação

O ITS-Demandas é uma aplicação **Next.js** (Node.js) que precisa de um processo rodando continuamente para atender requisições web, e de um banco de dados **PostgreSQL** — hoje provido pelo **Supabase**, um serviço gerenciado de banco de dados na nuvem.

Importante entender que são duas peças independentes:

- **A aplicação em si** (o código Next.js) — é isso que este guia ensina a colocar para rodar no servidor/VM da empresa.
- **O banco de dados** — continua no Supabase (nuvem), independentemente de onde a aplicação estiver rodando. Isso é assim porque o sistema foi construído usando os recursos do Supabase diretamente (autenticação de acesso ao banco, chaves de API); trazer o banco de dados também para dentro da empresa exigiria substituir o Supabase por um PostgreSQL autogerenciado e adaptar o código — não é o escopo deste guia.

NOTA

Isso significa que o servidor/VM da empresa precisa de acesso de saída à internet (para conversar com o Supabase), mesmo que o acesso de entrada (quem acessa o sistema) fique restrito à rede interna.

1.3 Opções de implantação

Este guia cobre em detalhe a opção de **servidor/VM próprio** (linux, com Nginx e PM2), que é o cenário corporativo típico, e apresenta também, no Capítulo 9, uma alternativa em **contêiner Docker** para equipes que já usam essa tecnologia.

TAMBÉM É POSSÍVEL USAR A VERCEL

O sistema foi originalmente desenhado para deploy simplificado na Vercel (plataforma criadora do Next.js), que faz todo o provisionamento de servidor automaticamente. Essa opção é mais simples e rápida, mas depende de um serviço externo — por isso este guia foca no cenário de servidor próprio, que é o que foi solicitado para o ambiente corporativo.

2. Pré-requisitos

Antes de começar, reúna o que será necessário. Ter tudo isso em mãos evita interrupções no meio do processo.

2.1 Hardware recomendado para a VM

Recurso	Mínimo	Recomendado
vCPU	1	2
Memória RAM	1 GB	2 GB
Armazenamento	15 GB	25 GB (SSD)
Sistema operacional	Ubuntu Server 22.04 LTS ou 24.04 LTS (64 bits)	

O ITS-Demandas é uma aplicação leve — esses números atendem confortavelmente uma empresa de porte pequeno a médio. Para uso mais intenso (centenas de usuários simultâneos), considere 4 GB de RAM e 2-4 vCPUs.

2.2 Software necessário

Software	Versão	Finalidade
Node.js	20.x LTS ou superior	Executar a aplicação Next.js
npm	Incluso no Node.js	Instalar as dependências do projeto
Git	Qualquer versão recente	Transferir/atualizar o código-fonte
PM2	Qualquer versão recente	Manter a aplicação rodando e reiniciar sozinha se cair
Nginx	Qualquer versão recente	Proxy reverso e HTTPS
Certbot	Qualquer versão recente	Certificado SSL gratuito (Let's Encrypt)

2.3 Rede e domínio

- Um **domínio ou subdomínio** apontando para o IP do servidor (ex.: `demandas.itsalvador.com.br`), configurado no DNS da empresa.

- Portas **80** (HTTP) e **443** (HTTPS) liberadas no firewall da rede/nuvem onde a VM está, para acesso externo.
- Se o acesso for só pela rede interna da empresa (sem exposição à internet), um registro DNS interno ou o IP direto já resolve — HTTPS com certificado público não se aplica nesse caso (ver observação no Capítulo 7).

2.4 Conta Supabase

Uma conta gratuita ou paga em supabase.com, com um projeto criado para o ITS-Demandas (coberto em detalhe no Capítulo 3).

3. Preparando o Banco de Dados

O banco de dados é configurado uma única vez, antes de colocar a aplicação no ar. Estes passos são feitos direto no painel do Supabase, pelo navegador — não precisam do servidor/VM ainda.

3.1 Criando o projeto no Supabase

1. Acesse supabase.com e crie uma conta (ou entre com uma já existente da empresa).
2. Clique em **New Project**, escolha um nome (ex.: `its-demandas-producao`) e uma senha forte para o banco — guarde essa senha em um cofre de senhas da empresa.
3. Escolha a região mais próxima fisicamente dos usuários (ex.: São Paulo, se disponível) para menor latência.
4. Aguarde alguns minutos até o projeto ficar pronto.

3.2 Rodando o schema

Com o projeto criado, vá em **SQL Editor** no painel do Supabase e rode o conteúdo do arquivo `supabase/schema.sql`, incluído no código-fonte do sistema. Esse arquivo cria todas as tabelas, índices e regras necessárias de uma só vez.

ATENÇÃO

Rode o `schema.sql` uma única vez, num banco vazio. Se precisar refazer do zero mais tarde, use primeiro o `supabase/drop_all.sql` (também incluso), que apaga todas as tabelas do sistema antes de recriar.

3.3 Coletando as chaves de API

Ainda no painel do Supabase, vá em **Project Settings** → **API** e anote dois valores, que serão usados como variáveis de ambiente da aplicação (Capítulo 6.2):

No painel do Supabase	Variável de ambiente
Project URL	<code>NEXT_PUBLIC_SUPABASE_URL</code>
service_role key (em "Project API keys")	<code>SUPABASE_SERVICE_ROLE_KEY</code>

SEGREDO CRÍTICO

A `service_role key` dá acesso total e irrestrito ao banco de dados, ignorando qualquer regra de segurança. Trate-a como uma senha de administrador: nunca a compartilhe, nunca a exponha no navegador, nunca a coloque em repositório de código público.

3.4 Criando o usuário administrador

O sistema inclui um script (`scripts/seed-admin.mjs`) que cria o primeiro usuário administrador diretamente no banco, já com a senha corretamente criptografada. Ele é executado depois que a aplicação estiver instalada no servidor (passo detalhado no Capítulo 6.3), com o comando:

```
npm run seed
```

4. Provisionando o Servidor

4.1 Escolhendo a VM

Pode ser uma VM em nuvem corporativa (Azure, AWS, Google Cloud, Oracle Cloud) ou uma máquina virtual dentro do próprio data center da empresa (VMware, Proxmox, Hyper-V). O procedimento a partir daqui é o mesmo, desde que o sistema operacional seja Linux Ubuntu conforme especificado no Capítulo 2.1.

4.2 Acesso inicial e hardening básico

Ao provisionar a VM, algumas boas práticas de segurança básicas antes de instalar qualquer coisa:

1. **Crie um usuário não-root** para operar o servidor no dia a dia:

```
adduser deploy
usermod -aG sudo deploy
```

2. **Configure acesso por chave SSH** (em vez de senha) para esse usuário, e desative login de senha e login direto como `root` no arquivo `/etc/ssh/sshd_config`:

```
PermitRootLogin no
PasswordAuthentication no
```

Depois, reinicie o serviço SSH: `sudo systemctl restart sshd`.

3. **Atualize o sistema** antes de prosseguir:

```
sudo apt update && sudo apt upgrade -y
```

5. Instalando Dependências no Servidor

Todos os comandos abaixo são executados no terminal do servidor, via SSH, já como o usuário não-root criado no capítulo anterior.

5.1 Node.js

Recomenda-se instalar via NodeSource, que mantém pacotes atualizados para Ubuntu:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt install -y nodejs  
  
# confirmar a instalação  
node -v # deve mostrar v20.x ou superior  
npm -v
```

5.2 Git

```
sudo apt install -y git  
git --version
```

5.3 PM2

O PM2 é um gerenciador de processos para Node.js: mantém a aplicação rodando em segundo plano, reinicia sozinho se ela cair, e reinicia automaticamente depois de uma reinicialização do servidor.

```
sudo npm install -g pm2
```

5.4 Nginx

```
sudo apt install -y nginx  
sudo systemctl enable nginx  
sudo systemctl start nginx
```

6. Implantando a Aplicação

6.1 Transferindo o código

Se o código estiver em um repositório Git da empresa (GitHub, GitLab, Bitbucket, ou um servidor Git interno):

```
cd /home/deploy
git clone <url-do-repositorio> its-demandas
cd its-demandas
```

Se o código for transferido manualmente (por exemplo, a partir do pacote entregue), copie a pasta inteira do projeto para o servidor via `scp` ou uma ferramenta de transferência segura equivalente:

```
scp -r its-demandas deploy@ip-do-servidor:/home/deploy/
```

6.2 Variáveis de ambiente

Crie o arquivo `.env.local` na raiz do projeto, com os valores reais coletados no Capítulo 3.3:

```
cd /home/deploy/its-demandas
nano .env.local
```

Conteúdo do arquivo:

```
NEXT_PUBLIC_SUPABASE_URL=https://SEU-PROJETO.supabase.co
SUPABASE_SERVICE_ROLE_KEY=sua-service-role-key-aqui
AUTH_SECRET=uma-string-longa-e-aleatoria
```

Gere um valor seguro para `AUTH_SECRET` com:

```
openssl rand -base64 48
```

ATENÇÃO

O arquivo `.env.local` contém segredos — garanta que ele não seja versionado no Git (o projeto já vem com um `.gitignore` que o exclui) e que só o usuário que roda a aplicação tenha permissão de leitura: `chmod 600 .env.local`.

6.3 Instalação e build

```
npm install
npm run build
```

Se este for o primeiro deploy, crie também o usuário administrador inicial:

```
npm run seed
```

6.4 Rodando com PM2

```
# inicia a aplicação e dá um nome ao processo
pm2 start npm --name its-demandas -- start

# confirma que está rodando
pm2 status

# configura o PM2 para iniciar junto com o servidor
pm2 startup
pm2 save
```

Por padrão, o Next.js sobe na porta **3000**. O Nginx (Capítulo 7) fica responsável por expor a aplicação nas portas 80/443 e repassar as requisições para a porta 3000 internamente.

6.5 Alternativa: serviço systemd

Para equipes que preferem não usar o PM2 e já têm um padrão de operação baseado em systemd, crie o arquivo `/etc/systemd/system/its-demandas.service`:

```
[Unit]
Description=ITS-Demandas
After=network.target

[Service]
Type=simple
User=deploy
WorkingDirectory=/home/deploy/its-demandas
ExecStart=/usr/bin/npm start
Restart=on-failure
RestartSec=5
EnvironmentFile=/home/deploy/its-demandas/.env.local

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable its-demandas
sudo systemctl start its-demandas
sudo systemctl status its-demandas
```

7. Proxy Reverso e HTTPS

O Nginx fica na frente da aplicação: recebe as requisições do navegador nas portas padrão (80/443) e repassa internamente para o processo Node.js na porta 3000, além de cuidar da criptografia HTTPS.

7.1 Configurando o Nginx

Crie o arquivo `/etc/nginx/sites-available/its-demandas` :

```
server {
    listen 80;
    server_name demandas.itsalvador.com.br;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Ative o site e recarregue o Nginx:

```
sudo ln -s /etc/nginx/sites-available/its-demandas /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

7.2 Certificado SSL com Let's Encrypt

Com o domínio já apontando para o servidor e a porta 80 acessível, gere o certificado gratuito com o Certbot:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d demandas.itsalvador.com.br
```

O Certbot configura automaticamente o HTTPS no Nginx e o redirecionamento de HTTP para HTTPS. Ele também instala uma renovação automática do certificado (que expira a cada 90 dias); confirme que está ativa com:

```
sudo systemctl status certbot.timer
```

SOMENTE REDE INTERNA, SEM DOMÍNIO PÚBLICO?

Se o sistema só será acessado de dentro da rede da empresa (sem exposição à internet), o Let's Encrypt não se aplica (ele exige validar um domínio publicamente acessível).

Nesse caso, utilize um certificado próprio emitido pela CA interna da empresa, ou opere apenas em HTTP dentro da rede interna confiável — avalie esse ponto com o time de segurança da informação.

8. Segurança

8.1 Firewall

```
sudo apt install -y ufw
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
sudo ufw status
```

Isso libera apenas SSH (22), HTTP (80) e HTTPS (443) — a porta 3000 (Node.js) não precisa ficar aberta externamente, já que só o Nginx (na mesma máquina) precisa alcançá-la.

8.2 Segredos e variáveis sensíveis

- Nunca commitar o arquivo `.env.local` em nenhum repositório.
- Restringir a leitura do arquivo só ao usuário que roda a aplicação (`chmod 600`).
- Guardar uma cópia da `service_role key` e do `AUTH_SECRET` em um cofre de senhas corporativo (não em planilhas ou mensagens).
- Rotacionar o `AUTH_SECRET` periodicamente invalida todas as sessões ativas — útil em caso de suspeita de vazamento.

8.3 Atualizações do sistema operacional

Mantenha o Ubuntu atualizado regularmente:

```
sudo apt update && sudo apt upgrade -y
```

Considere habilitar atualizações de segurança automáticas com `unattended-upgrades` .

8.4 Backups

- **Banco de dados:** o Supabase já faz backups automáticos diários nos planos pagos; confirme a política do plano contratado e, se necessário, configure backups adicionais manuais (`pg_dump`) para um armazenamento próprio da empresa.
- **Aplicação:** como o código-fonte fica em controle de versão (Git), o "backup" da aplicação em si é o próprio repositório. Garanta que o arquivo `.env.local` do servidor de produção também esteja salvo em um local seguro fora da VM (um cofre de segredos, por exemplo), já que ele não fica no Git.

9. Alternativa: Implantação com Docker

Para equipes que já padronizam a operação de aplicações em contêineres, esta alternativa substitui os Capítulos 5 e 6 (Node.js/PM2 direto no servidor) por um contêiner Docker. O Nginx e o Certbot (Capítulo 7) continuam necessários da mesma forma, na própria VM host.

Crie um arquivo `Dockerfile` na raiz do projeto:

```
FROM node:20-alpine AS base

FROM base AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM base AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM base AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next ./next
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./package.json

EXPOSE 3000
CMD ["npm", "start"]
```

E um `docker-compose.yml` para simplificar a execução:

```
services:
  its-demandas:
    build: .
    restart: unless-stopped
    ports:
      - "3000:3000"
    env_file:
      - .env.local
```

Para construir e subir o contêiner:

```
docker compose up -d --build
```

```
# ver logs
```

```
docker compose logs -f
```

```
# rodar o script de criação do administrador dentro do contêiner
```

```
docker compose exec its-demandas npm run seed
```

DICA

Com Docker, atualizar a aplicação é: puxar o código novo (`git pull`), rodar `docker compose up -d --build` de novo, e o Docker recria o contêiner com a versão nova, praticamente sem tempo de parada perceptível.

10. Processo de Atualização

Sempre que uma nova versão do código for entregue (correções ou funcionalidades novas), o processo de atualização segue os mesmos passos, de forma resumida.

Com PM2

```
cd /home/deploy/its-demandas
git pull                # ou: substituir os arquivos pela nova versão
npm install             # caso dependências tenham mudado
npm run build
pm2 restart its-demandas
```

Com systemd

```
cd /home/deploy/its-demandas
git pull
npm install
npm run build
sudo systemctl restart its-demandas
```

Rollback (voltar para a versão anterior)

Se algo der errado após uma atualização, e o código estiver em Git:

```
git log --oneline        # identificar o commit anterior
git checkout <hash-do-commit-anterior>
npm install
npm run build
pm2 restart its-demandas # ou systemctl restart its-demandas
```

ATENÇÃO COM MUDANÇAS DE BANCO DE DADOS

Se a atualização incluir um novo arquivo de migração SQL (ex.: `supabase/migration_00X_*.sql`), rode esse arquivo no SQL Editor do Supabase **antes** de reiniciar a aplicação com o código novo — e saiba que um rollback do código não desfaz uma migração de banco já aplicada.

11. Monitoramento e Manutenção

Logs

```
# logs da aplicação (PM2)
pm2 logs its-demandas

# logs de acesso e erro do Nginx
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```

Verificação de saúde

Uma checagem simples de que a aplicação está respondendo:

```
curl -I http://localhost:3000
```

Deve retornar `HTTP/1.1 200 OK` (ou um redirecionamento para a tela de login).

Reinício automático

O PM2 já reinicia a aplicação sozinho se o processo cair (`pm2 startup` + `pm2 save` , feito no Capítulo 6.4, garante isso mesmo depois de reiniciar o servidor). Para systemd, a linha `Restart=on-failure` do arquivo de serviço (Capítulo 6.5) cumpre o mesmo papel.

12. Checklist Pós-Implantação

Antes de considerar a implantação concluída, confirme cada item abaixo.

- ☐ O schema do banco foi rodado no Supabase sem erros.
- ☐ As variáveis de ambiente (`.env.local`) estão configuradas com os valores reais.
- ☐ `npm run build` concluiu sem erros.
- ☐ A aplicação responde em `http://localhost:3000` dentro do servidor.
- ☐ O Nginx está repassando corretamente para a porta 3000.
- ☐ O domínio configurado abre o sistema pelo navegador, de fora do servidor.
- ☐ O certificado HTTPS está ativo e o cadeado aparece no navegador (se aplicável ao cenário).
- ☐ O usuário administrador inicial foi criado (`npm run seed`) e o login funciona.
- ☐ A senha inicial do administrador foi trocada pela própria tela de Usuários.
- ☐ O firewall (ufw) está ativo, liberando só as portas necessárias.
- ☐ O PM2 (ou systemd) está configurado para reiniciar a aplicação automaticamente após reboot do servidor.
- ☐ Existe um plano de backup documentado (banco de dados e arquivo de variáveis de ambiente).
- ☐ Pelo menos uma demanda de teste foi criada, movida no Kanban e excluída com sucesso.

13. Solução de Problemas

Sintoma	Causa provável	O que fazer
Tela em branco ou erro 502 Bad Gateway	A aplicação Node.js não está rodando, ou caiu	Rodar <code>pm2 status</code> (ou <code>systemctl status its-demandas</code>) e checar os logs
Erro de login mesmo com credenciais corretas	<code>AUTH_SECRET</code> mudou, invalidando sessões antigas — ou o usuário está inativo	Confirmar a variável de ambiente e o status do usuário na tela de Usuários
Erro relacionado a "Supabase" ou "fetch failed" nos logs	Variáveis <code>NEXT_PUBLIC_SUPABASE_URL</code> / <code>SUPABASE_SERVICE_ROLE_KEY</code> erradas, ou servidor sem acesso à internet	Conferir as variáveis e testar <code>curl https://SEU-PROJETO.supabase.co</code> do próprio servidor
Certificado HTTPS não gera / expira	Domínio não aponta para o servidor, ou porta 80 bloqueada	Confirmar DNS e firewall antes de rodar o Certbot novamente
<code>npm run build</code> falha	Versão do Node.js incompatível, ou dependência corrompida	Confirmar <code>node -v</code> (20+) e rodar <code>rm -rf node_modules package-lock.json && npm install</code>
Mudanças no código não aparecem depois do deploy	Esqueceu de rodar <code>npm run build</code> e/ou reiniciar o processo	Repetir o processo de atualização (Capítulo 10) na ordem certa

14. Anexo: Referência de Variáveis de Ambiente

Variável	Obrigatória	Descrição
NEXT_PUBLIC_SUPABASE_URL	Sim	URL do projeto Supabase (Project Settings → API)
SUPABASE_SERVICE_ROLE_KEY	Sim	Chave de acesso total ao banco — segredo crítico, nunca exposto ao navegador
AUTH_SECRET	Sim	String aleatória usada para assinar as sessões de login (JWT)
SEED_ADMIN_SENHA	Não	Usada apenas pelo script <code>npm run seed</code> , para definir a senha inicial do administrador; se omitida, usa um valor padrão do script
NODE_ENV	Não	Definida automaticamente como <code>production</code> pelo <code>npm run build / npm start</code>